NAME:ABHINAV CHUGH
SRN:PES2UG24AM007
GIT HUB REPO
LINK:https://github.com/abhi1207-chugh/PES2UG24AM007-pes-VCS/tree/main

PHASE 1A:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ make clean
make test_objects
./test_objects
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

PHASE1B:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

PHASE 2A:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

PHASE 2B:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ make
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'build_tree':
tree.c:205:9: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
  205 |     if (object_write(OBJ_TREE, data, len, out_id) != 0) {
      |         ^~~~~~~~~~~
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ ./pes init
Initialized empty PES repository in .pes/
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ echo "hello world" > file.txt
./pes add file.txt
./pes commit -m "first commit"
error: failed to load index
error: commit failed
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ find .pes/objects -type f
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$
```

PHASE 3A

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ ./pes init
echo "hello world" > file.txt
./pes add file.txt
cat .pes/index
Initialized empty PES repository in .pes/
304444 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776444298 12 file.txt
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$
```

PHASE 3B

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ find .pes/objects -type f
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

PHASE 4A:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ ./pes init
echo "hello world" > file.txt
./pes add file.txt
./pes commit -m "first commit"
Initialized empty PES repository in .pes/
Committed: 9925de7b760c... first commit
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$
```

PHASE 4B:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ cat .pes/HEAD
ref: refs/heads/main
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$
```

PHASE 4C:

```
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$ find .pes/objects -type f
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/99/25de7b760c11659626d570fb9da4b19c01ae90afb5f57e6887fbfbc8990192
.pes/objects/f2/dce82ed163c4bfc3174a5d5594144ba34a0b664fca18fe2ac3baeeffdcc7f0
.pes/objects/42/1412b3c9daf5a29ede2cc56a788e7261ffd9b2c2f2e4b787c01e3d29a83955
abhinav@abhinav:~/PES2UG24AM007-pes-VCS$
```

Q5.1 — Checkout Implementation
Read .pes/refs/heads/<branch> to get target commit hash
Read that commit's tree recursively
Update working directory files to match target tree (write/delete files)
Update .pes/HEAD to ref: refs/heads/<branch>
Update .pes/index to match new tree
Complexity: must handle file deletions (files in current tree but not target), directory creation/removal, and permission changes.

Q5.2 — Dirty Working Directory Detection
For each file in index: stat() it and compare mtime + size to index entry
If mismatch → file modified in working dir
If target branch has different blob hash for same file → conflict
Refuse checkout if both are true (local modified + branch differs)
No re-hashing needed; metadata comparison is fast.

Q5.3 — Detached HEAD
In detached HEAD, new commits exist in object store but no branch file points to them. If you switch away, commits become unreachable (orphaned). Recovery: run find .pes/objects -type f to list all objects, use object_read to find commit objects, reconstruct the chain manually, then create a branch pointing to the orphaned commit: echo <hash> > .pes/refs/heads/recovered

Q6.1 — Garbage Collection Algorithm
1.Start from all branch refs + HEAD → get root commit hashes
2.BFS/DFS: for each commit, add its hash → read commit → add tree hash → recurse into trees → add blob hashes
3.Use a HashSet (hash table of ObjectID) for O(1) membership test
4.Walk all files in .pes/objects/ — delete any not in reachable set
Estimate for 100k commits, 50 branches:

Avg 10 files/commit = ~1M blob+tree objects
Must visit all ~1M objects in reachability pass
Total object visits ≈ 1–2M

Q6.2 — GC Race Condition
GC scans reachable objects — finds object X is unreachable → marks for deletion
Concurrent commit writes new tree referencing object X → writes commit
GC deletes object X before commit finishes
Repository now has commit pointing to deleted object → corruption

Git avoids this by:

Using a grace period (objects newer than 14 days never deleted)
Lock files during ref updates
Two-phase GC: mark phase never deletes, sweep only after full scan